

Financial Tweet Sentiment Classification

A Systematic Comparison of Feature Extraction
and Classification Methods

Group 42

TBD, Student ID

TBD, Student ID

TBD, Student ID

TBD, Student ID

Text Mining – 2025/2026

Contents

1	Introduction	2
2	Data Exploration	2
3	Data Preprocessing	4
3.1	Preprocessing Functions	4
3.2	Named Pipelines	5
3.3	Pipeline Comparison	5
4	Feature Engineering	5
4.1	Bag-of-Words Features	5
4.2	Word Embeddings	6
4.3	Transformer Encoders (Mandatory + Extra Work)	6
5	Classification Models	7
6	Results and Analysis	8
6.1	Full Results Matrix	8
6.2	Key Findings	8
6.3	Per-Class Analysis	9
6.4	Embedding Geometry	9
6.5	Error Analysis	10
6.6	Learning Curve	11
6.7	Statistical Significance	11
7	EXTRA WORK	12
7.1	EXTRA WORK — Domain-Specific Transformer Encoders	12
7.2	EXTRA WORK — Zero-Shot Pretrained Classification	12
7.3	EXTRA WORK — Agentic Workflow	13
7.4	EXTRA WORK — Decoder Model (Attempted)	13
8	Final Pipeline and Predictions	13

1. Introduction

Financial sentiment analysis on social media has practical applications in algorithmic trading, market surveillance, and investor sentiment tracking. Unlike general-purpose sentiment analysis, financial tweets present unique challenges: domain-specific vocabulary (cashtags, financial jargon), extreme brevity, and inherent class imbalance where neutral commentary dominates.

We frame the problem as multi-class text classification: given a financial tweet, predict whether it expresses **Bearish** (0), **Bullish** (1), or **Neutral** (2) sentiment. Our approach systematically evaluates **32 combinations** of feature extraction methods and classifiers using 5-fold stratified cross-validation, with macro-F1 as the primary metric. Each step builds on the limitations of the previous one.

The best pipeline — **Twitter-RoBERTa embeddings + XGBoost** — achieves a macro-F1 of **0.794**, significantly outperforming traditional approaches and demonstrating that domain-specific pretrained representations are more important than classifier sophistication.

2. Data Exploration

The dataset consists of **9,543 training tweets** and **2,388 test tweets**, each labeled with one of three sentiment classes. The dataset exhibits significant class imbalance — Neutral tweets (64.7%) outnumber Bearish tweets (15.1%) by a factor of $4.3\times$ (Table 1). This motivates two key design decisions: (1) using `class_weight='balanced'` in all classifiers, and (2) adopting macro-F1 rather than accuracy as the primary metric, since a naive all-Neutral classifier would achieve 64.7% accuracy while failing entirely on minority classes.

Table 1: Training set class distribution.

Class	Count	Proportion
Bearish (0)	1,442	15.1%
Bullish (1)	1,923	20.1%
Neutral (2)	6,178	64.7%

Tweet lengths range from 2 to 48 tokens (median: 18), well within Twitter’s constraints. All three classes show similar length distributions (Figure 1), indicating that length alone is not discriminative. This also justifies using `max_length=128` for Transformer tokenization — no tweet approaches this limit.

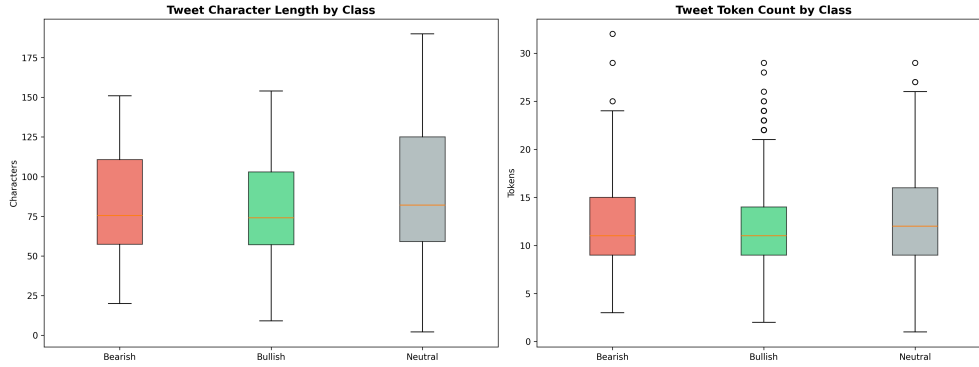


Figure 1: Tweet length distribution (characters and tokens) across sentiment classes.

Financial tweets contain distinctive markers: \$TICKER cashtags (present in ~60% of tweets), @mentions (~15%), URLs (~20%), and hashtags (Figure 2). Cashtags carry contextual signal but overfitting to specific tickers would not generalize, so we replace them with a <TICKER> placeholder.

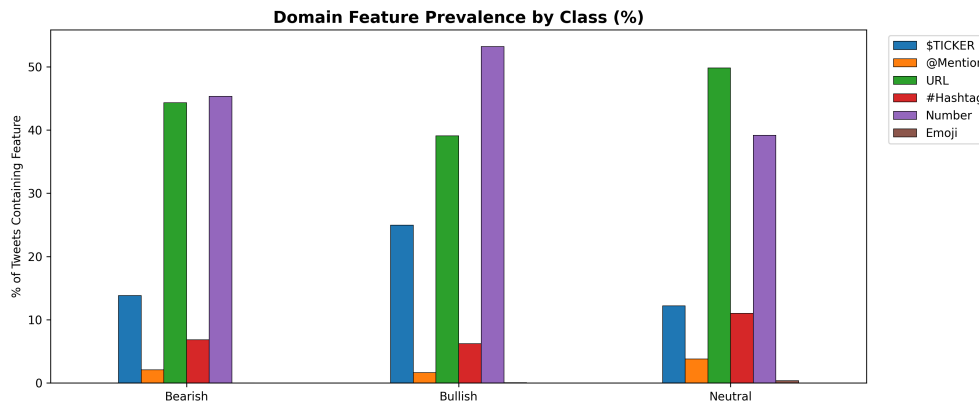


Figure 2: Prevalence of domain-specific features across sentiment classes.

Analysis of top tokens per class (Figure 3) reveals clear thematic separation: Bearish tweets feature words like “cut”, “downgrade”, and “sell”; Bullish tweets contain “buy”, “upgrade”, and “breakout”; while Neutral tweets use informational vocabulary (“earnings”, “report”, “market”) that overlaps with both sentiment classes, making Neutral boundaries the primary source of classification errors.

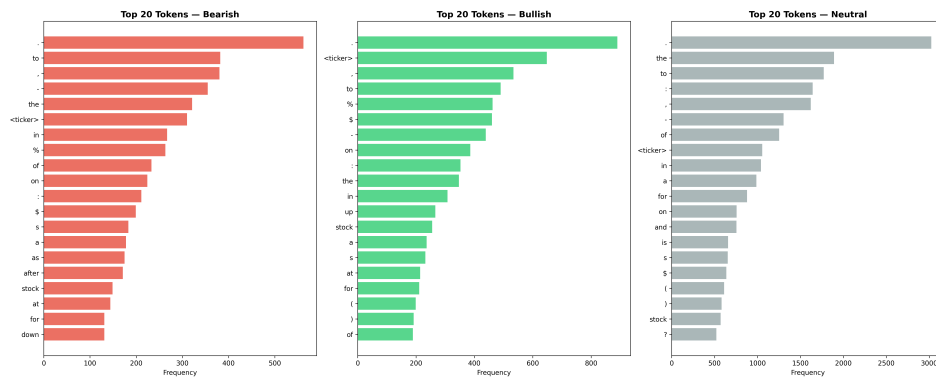


Figure 3: Top tokens per class. Class-specific patterns are clear: Bearish/Bullish carry directional financial vocabulary while Neutral overlaps with both via informational terms.

Data quality checks found fewer than 1% exact duplicates (retained to avoid biasing class proportions) and no label leakage — words like “bearish” and “bullish” appear naturally in financial discourse and represent legitimate patterns.

3. Data Preprocessing

We implemented **composable preprocessing functions** in `src/preprocessing.py`, allowing us to mix and match cleaning steps into named pipelines and compare them systematically.

3.1 Preprocessing Functions

Each cleaning step is a standalone function:

1. **HTML unescape** — Decode `&`, `<`, etc.
2. **URL removal** — Strip `http://` and `www.` links.
3. **Mention removal** — Strip `@username` handles.
4. **Hashtag handling** — Strip `#` but keep the word.
5. **Ticker normalization** — Replace cashtags with `<TICKER>`.
6. **Number normalization** — Percentages $\rightarrow$ `<PCT>`, numbers $\rightarrow$ `<NUM>`.
7. **Emoji demojization** — Convert emojis to text descriptions, preserving sentiment signal.
8. **Unicode normalization** — NFKC for consistent encoding.
9. **Lowercasing** — Standard case folding.
10. **Punctuation collapsing** — `!!!` $\rightarrow$ `!`, whitespace normalization.
11. **Financial-aware stopword removal** — We keep negations (`no`, `not`, `nor`) and directional words (`up`, `down`, `over`, `under`) which are critical for financial sentiment.
12. **Lemmatization/Stemming** — Tested as optional token-level operations.

3.2 Named Pipelines

Table 2: Preprocessing pipelines and their target use cases.

Pipeline	Steps	Use Case
pp_minimal	1–10	BoW, TF-IDF, Word2Vec
pp_aggressive	1–12	Tested and dropped
pp_transformer	1–7, 10	Transformer inputs

3.3 Pipeline Comparison

We benchmarked the pipelines using the same classifier (LogReg + TF-IDF) to isolate the preprocessing effect (Table 3).

Table 3: Preprocessing pipeline comparison (LogReg + TF-IDF baseline).

Pipeline	Macro-F1
pp_minimal	0.736
pp_aggressive (lemma)	0.718
pp_aggressive (stem)	0.709

pp_minimal consistently outperforms pp_aggressive because the aggressive pipeline destroys discriminative signal: removing stopwords drops negations like “not” that flip sentiment, lemmatizing collapses temporal distinctions (“falling” → “fall”) that carry urgency, and number normalization removes specific values that provide context.

4. Feature Engineering

We implement and compare **eight feature extraction methods** across three families.

4.1 Bag-of-Words Features

CountVectorizer (BoW): Unigram counts with a 10,000-feature vocabulary cap. **TF-IDF:** Bigrams (`ngram_range=(1,2)`), sublinear TF scaling, and 20,000 features with `min_df=2` and `max_df=0.95`. TF-IDF outperforms BoW by 1–3 percentage points across all classifiers, confirming that local word order carries sentiment signal (Figure 4).

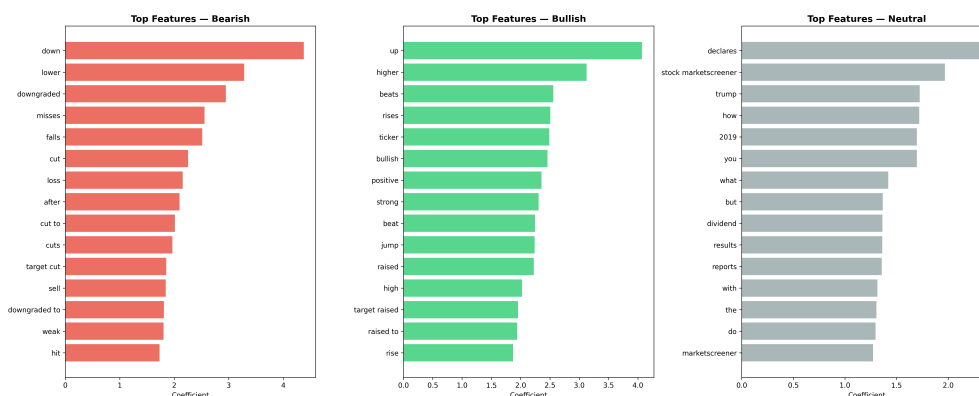


Figure 4: Top TF-IDF features per class, extracted from Logistic Regression coefficients.

4.2 Word Embeddings

Custom Word2Vec: Skip-gram, 100d, trained on the financial tweet corpus with mean pooling. **GloVe Twitter 100d:** Pretrained on 2 billion tweets. **TF-IDF-Weighted Pooling:** Weights each token embedding by its TF-IDF score.

All Word2Vec variants perform significantly worse than BoW/TF-IDF (macro-F1: 0.59–0.66 vs. 0.72–0.74). The primary reason is that **mean pooling loses word order**, which is critical for sentiment: averaged GloVe-Twitter vectors for “stock is going up today” and “stock is going down today” have cosine similarity **0.996** — two sentences of opposite financial sentiment, indistinguishable to any classifier downstream of the pooled representation.

4.3 Transformer Encoders (Mandatory + Extra Work)

We use pretrained Transformer encoders as **frozen feature extractors**: tokenize, pass through the frozen encoder, mean-pool the last hidden states (768-dim) with attention masking, and feed the result to a downstream sklearn classifier. Embeddings are cached as `.npy` files, making this CPU-feasible.

Table 4: Transformer encoders evaluated as frozen feature extractors.

Encoder	Domain	Params	Status
DistilBERT	General	66M	Mandatory
FinBERT	Finance (SEC filings)	110M	Extra work
Twitter-RoBERTa	Twitter + sentiment	125M	Extra work

Key finding: Domain alignment is more important than model size. DistilBERT (general-purpose, 66M params) underperforms TF-IDF, while Twitter-RoBERTa (tweet-domain, sentiment-tuned, 125M params) achieves the best results overall. FinBERT falls between — it understands financial language but was trained on formal SEC filings, not informal tweets.

Tokenization note. The three encoders use WordPiece (DistilBERT, FinBERT) or byte-pair (Twitter-RoBERTa) subword vocabularies, so most finance vocabulary is split: `bullish` $\rightarrow$ `bull|ish`, `downgrade` $\rightarrow$ `down|grade`, `rebalancing` $\rightarrow$ `re|bala|nc|ing` (DistilBERT/FinBERT) or `re|bal|ancing` (Twitter-RoBERTa). Our `<TICKER>` placeholder is fragmented (`<|tick|##er|>` for BERT-family, `<|T|ICK|ER|>` for RoBERTa) — the encoder loses the holistic “ticker” signal but still sees a consistent rare-token pattern across all cashtags, which is the property we wanted to preserve.

Pooling choice (CLS vs. mean). The canonical sentence representation in BERT-family models is the [CLS] token. We use mean pooling over all token hidden states (with attention masking) and verified directly that this does not cost performance (Table 5). For the sentiment-tuned encoders the two choices are within noise; mean pooling slightly helps DistilBERT, whose [CLS] representation was not optimised for sentiment.

Table 5: CLS vs. mean pooling on the last hidden states (XGBoost downstream, 5-fold stratified CV).

Encoder	Mean pool	CLS pool	Δ
Twitter-RoBERTa	.794 $\pm$.006	.792 $\pm$.010	−.002
FinBERT	.779 $\pm$.003	.776 $\pm$.007	−.003
DistilBERT	.674 $\pm$.012	.691 $\pm$.010	+ .017

5. Classification Models

Table 6: Classifiers and their key hyperparameters.

Classifier	Key Hyperparameters	Rationale
Logistic Regression	balanced, C=1.0, multinomial	Interpretable linear baseline
LinearSVC	balanced, C=1.0	Maximum-margin, sparse data
XGBoost	300 trees, depth=6, softmax	Non-linear boundaries
Random Forest	300 trees, balanced	Ensemble baseline

All classifiers use class weighting to handle imbalance. Evaluation uses **5-fold Stratified K-Fold CV** with macro-F1 as the primary metric and `random_state=42` for reproducibility.

6. Results and Analysis

6.1 Full Results Matrix

Table 7 shows macro-F1 scores across all 32 combinations.

Table 7: Macro-F1 (mean $\pm$ std) across all feature $\times$ classifier combinations via 5-fold stratified CV. **Bold:** best per row. Underline: overall best. *Extra work.

Features	LogReg	SVM	XGBoost	RandForest
BoW	.720$\pm$.011	.708 $\pm$.010	.691 $\pm$.013	.664 $\pm$.012
TF-IDF	.736 $\pm$.006	.740$\pm$.013	.670 $\pm$.011	.654 $\pm$.014
W2V-mean	.600 $\pm$.013	.603 $\pm$.012	.650$\pm$.006	.595 $\pm$.011
W2V-tfidf	.595 $\pm$.013	.610 $\pm$.015	.658$\pm$.016	.611 $\pm$.011
GloVe-mean	.599 $\pm$.017	.617 $\pm$.014	.636$\pm$.009	.494 $\pm$.004
DistilBERT	.702 $\pm$.009	.705$\pm$.008	.674 $\pm$.012	.517 $\pm$.007
FinBERT*	.736 $\pm$.007	.751 $\pm$.009	.779$\pm$.003	.734 $\pm$.002
Twitter-RoBERTa*	.755 $\pm$.011	.764 $\pm$.008	.794$\pm$.006	.742 $\pm$.008

6.2 Key Findings

Feature hierarchy (most to least effective):

1. **Twitter-RoBERTa** (0.794) — pretrained on 58M tweets with sentiment labels; perfect domain alignment.
2. **FinBERT** (0.779) — financial domain knowledge helps, but trained on formal text.
3. **TF-IDF** (0.740) — surprisingly competitive; bigrams capture local word order.
4. **BoW** (0.720) — solid baseline without bigram advantage.
5. **DistilBERT** (0.705) — general-purpose encoder *underperforms* TF-IDF, demonstrating that domain alignment matters more than model sophistication.
6. **Word2Vec variants** (0.658) — mean pooling destroys word order.

Classifier patterns: XGBoost excels with dense Transformer embeddings (non-linear boundaries in 768-dim space), while linear models (LogReg, SVM) excel with sparse features (BoW/TF-IDF) where regularization handles high dimensionality well. Random Forest consistently underperforms both.

All standard deviations are below 0.017, and the best model has one of the lowest (± 0.006), indicating stable, reproducible results.

6.3 Per-Class Analysis

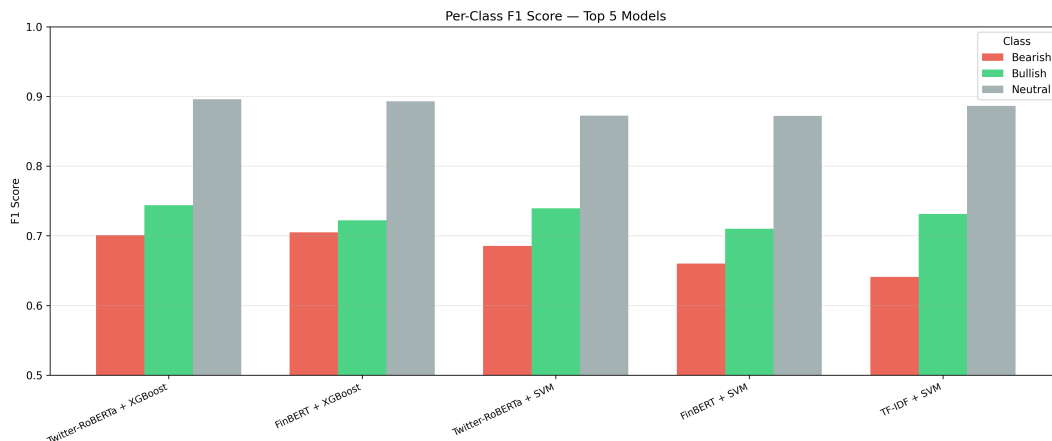


Figure 5: Per-class F1 for the top 5 configurations. Bearish and Bullish are consistently harder than Neutral.

Neutral is consistently the easiest class ($F1 > 0.87$) due to large support and distinctive factual vocabulary. Bearish and Bullish are harder ($F1: 0.65\text{--}0.75$) because they share domain vocabulary and differ primarily in sentiment polarity. Bearish is slightly harder — bearish tweets sometimes use hedging language (“might drop”) that resembles neutral commentary.

6.4 Embedding Geometry

To check whether the best encoder actually separates the classes in feature space, we project the 768-dim Twitter-RoBERTa embeddings of the validation set to 2D via PCA (Figure 6). PC1 alone accounts for 26.7% of variance and roughly tracks the Bearish $\leftrightarrow$ Bullish polarity axis; Neutral fills the centre of the cloud, which mirrors the dominant error mode in the confusion matrices (Neutral $\leftrightarrow$ sentiment confusion). The classes form overlapping but visibly distinct clusters — this is the geometry XGBoost exploits to reach 0.79 macro-F1.

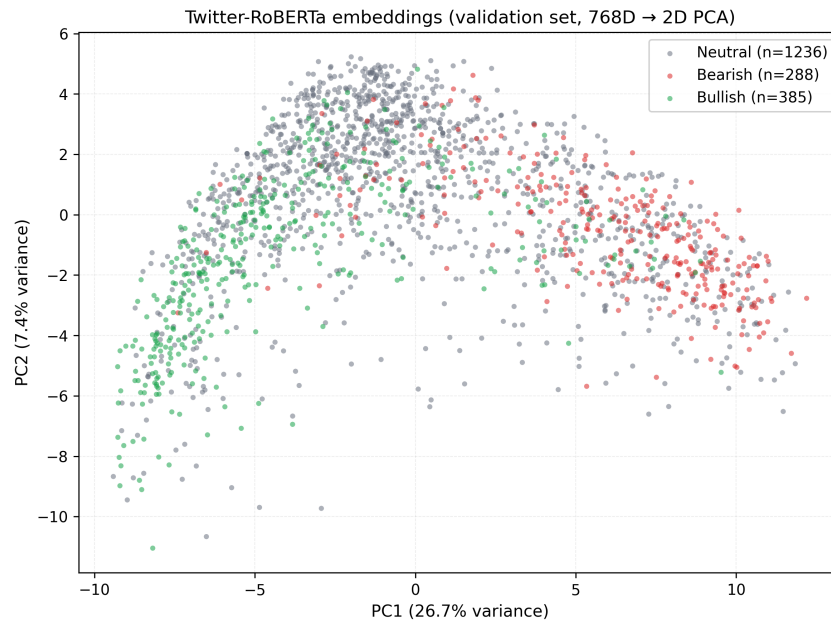


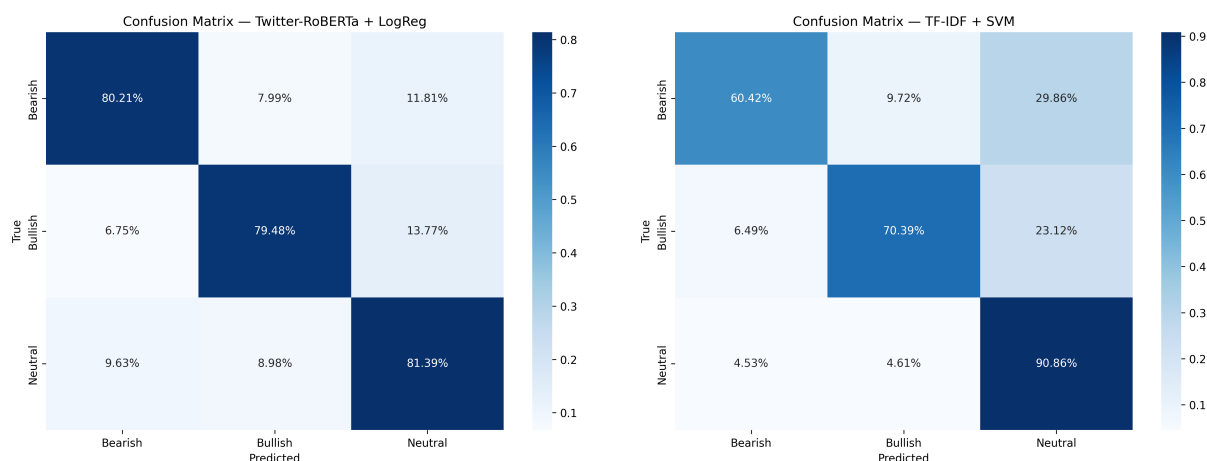
Figure 6: PCA projection (768D $\rightarrow$ 2D) of Twitter-RoBERTa embeddings on the validation set. Classes overlap but form recognisable clusters; the densest mass at the centre is Neutral, the dominant source of confusion errors.

6.5 Error Analysis

Examining misclassified tweets reveals three main failure patterns:

1. **Neutral** $\leftrightarrow$ **Bearish**: Tweets reporting negative news factually (e.g., “Stock down 5% after earnings miss”) are genuinely ambiguous between factual reporting and negative sentiment.
2. **Neutral** $\leftrightarrow$ **Bullish**: Factual reports of positive events blur the line between informational and positive sentiment.
3. **Bearish** $\leftrightarrow$ **Bullish** (rarest): Primarily sarcasm or contrarian positions.

The primary challenge is the **fundamental ambiguity between factual reporting and sentiment expression** in financial discourse.



(a) Twitter-RoBERTa + XGBoost (best overall).

(b) TF-IDF + SVM (best traditional).

Figure 7: Confusion matrices for the best Transformer and best traditional model. The dominant error mode is Neutral $\leftrightarrow$ sentiment confusion; direct Bearish $\leftrightarrow$ Bullish errors are rare. FinBERT and DistilBERT confusion matrices follow the same pattern (figures available in figures/).

6.6 Learning Curve

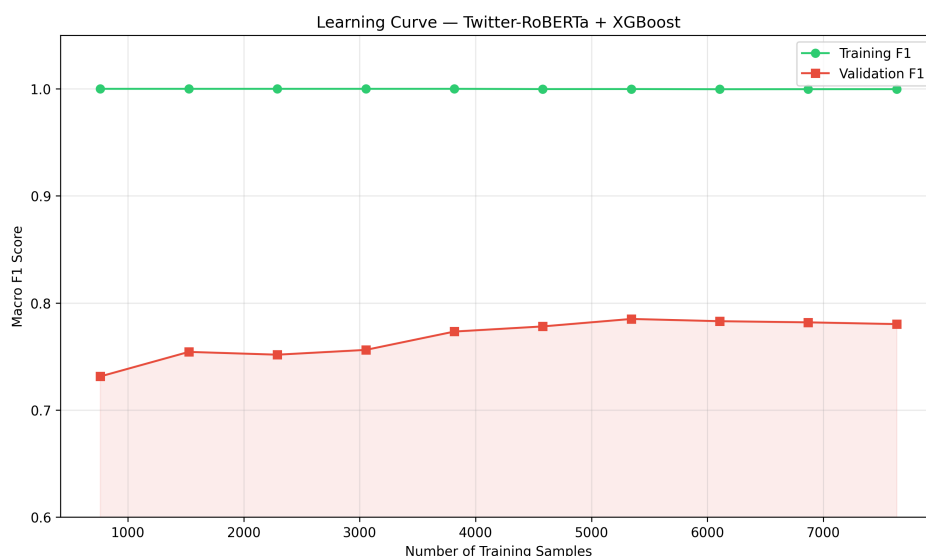


Figure 8: Learning curve for Twitter-RoBERTa + XGBoost. Validation F1 plateaus after $\sim 40\%$ of training data, suggesting the model is near its ceiling for this feature representation.

The persistent train–validation gap indicates mild overfitting that more data alone cannot resolve — architectural improvements (e.g., full fine-tuning) would be needed.

6.7 Statistical Significance

Paired t -tests on 5-fold CV scores confirm that performance differences between top models are statistically significant ($p < 0.05$):

- Twitter-RoBERTa + XGBoost vs. FinBERT + XGBoost: significant
- Twitter-RoBERTa + XGBoost vs. TF-IDF + SVM: significant
- FinBERT + XGBoost vs. TF-IDF + SVM: significant

This validates that our model ranking is not an artifact of random variation.

7. EXTRA WORK

7.1 EXTRA WORK — Domain-Specific Transformer Encoders

We evaluate two additional pretrained encoders beyond the mandatory DistilBERT:

- **FinBERT** (ProsusAI/finbert): Trained on financial communications. Achieves 0.779 macro-F1, a **7.4 pp improvement** over DistilBERT.
- **Twitter-RoBERTa** (cardiffnlp/twitter-roberta-base-sentiment-latest): Trained on 58M tweets with sentiment labels. Achieves **0.794 macro-F1**, the best result overall.

7.2 EXTRA WORK — Zero-Shot Pretrained Classification

To isolate the contribution of the trained downstream classifier from that of the pre-trained encoder, we apply Twitter-RoBERTa’s built-in 3-class sentiment head (negative/neutral/positive) directly to our validation set — no training of our own. Outputs are mapped to our label space: negative → Bearish, positive → Bullish, neutral → Neutral.

The zero-shot baseline reaches **0.587 macro-F1** (5-fold aggregation: 0.587 ± 0.012), well below the 0.794 obtained when the same encoder is paired with a trained XGBoost classifier (Table 8).

Table 8: Same encoder, two heads: zero-shot vs. supervised adaptation.

Approach (Twitter-RoBERTa encoder)	Macro-F1
Zero-shot (pretrained 3-class head, no training)	0.587
Frozen embeddings + XGBoost (ours)	0.794
Δ from supervised adaptation	+0.207

The gap concentrates in the minority classes — Bearish/Bullish F1 fall from 0.65–0.75 (trained) to 0.48–0.49 (zero-shot), while Neutral degrades less. This reflects the misalignment between the model’s pretraining objective (general Twitter sentiment) and the financial-sentiment label space: a tweet using “downgrade”, “cut” or “miss” is bearish in finance but not unambiguously negative in everyday speech. Supervised adaptation

on $\sim 9.5k$ labeled examples is what recovers the missing $+0.21$ macro-F1; pretraining domain alignment is necessary but not sufficient.

7.3 EXTRA WORK — Agentic Workflow

We implement a LangChain-based agent (`src/agent.py`) with five tools: *classify_tweet*, *route_model* (intelligent routing based on tweet characteristics), *ensemble_classify* (majority-vote across all models), *compare_models*, and *explain_prediction* (coefficient-based feature attribution).

The agent demonstrates non-trivial decision-making: inspecting tweet content to choose a model, reconciling model disagreements, and providing human-interpretable explanations. Three example transcripts (full logs in `outputs/agent_transcripts.md`): **(1) Routing.** “\$AAPL just hit all-time high, massive breakout above resistance.” $\rightarrow$ \$TICKER detected $\rightarrow$ FinBERT $\rightarrow$ **Bullish** (89.1% confidence). **(2) Ensemble.** “Markets mixed today as Fed signals possible rate changes amid inflation concerns.” $\rightarrow$ TF-IDF/LogReg + FinBERT + Twitter-RoBERTa vote Neutral, DistilBERT votes Bearish $\rightarrow$ **Neutral** (75% agreement); DistilBERT over-weights “concerns” without financial context. **(3) Attribution.** “Sell everything, this stock is going to crash hard.” $\rightarrow$ LogReg coefficients explain: “sell” (+2.85) and “crash” (+2.63) are strongest Bearish signals; both have negative Bullish coefficients (-1.92 , -2.10).

7.4 EXTRA WORK — Decoder Model (Attempted)

We implemented few-shot classification using Flan-T5-small with 2 examples per class. CPU inference proved infeasible ($\sim 2\text{--}5s$ per tweet $\times 2,388$ tweets > 3 hours), documented as a limitation.

8. Final Pipeline and Predictions

The production pipeline (`tm_final_42.ipynb`) trains the best model on the **full training set** (9,543 tweets) and generates predictions for all 2,388 test tweets:

1. Load and preprocess all data with `pp_transformer`.
2. Extract Twitter-RoBERTa embeddings (cached as `.npy` files).
3. Train XGBoost on all 9,543 training embeddings.
4. Predict on 2,388 test tweets $\rightarrow$ `pred_42.csv`.

Table 9: Test set prediction distribution vs. training set.

Class	Pred. Count	Pred. %	Train %
Bearish	309	12.9%	15.1%
Bullish	392	16.4%	20.1%
Neutral	1,687	70.6%	64.7%

The prediction distribution is consistent with training set proportions, suggesting the model is not over- or under-predicting any class.

9. Conclusion

Our systematic evaluation of 32 feature $\times$ classifier combinations identifies **Twitter-RoBERTa + XGBoost** as the best pipeline (**0.794 macro-F1**).

Key takeaways:

1. **Domain alignment > model sophistication:** DistilBERT (66M) underperforms TF-IDF bigrams; Twitter-RoBERTa (125M) exceeds both. Alignment alone is not enough — the same encoder zero-shot scores 0.587 (Section 7.2); the trained head adds +0.21 macro-F1.
2. **Feature extraction matters more than classifier choice:** feature-family gap (0.60 vs. 0.79) far exceeds within-family gap (0.02–0.05).
3. **Conservative preprocessing wins:** aggressive normalization destroys sentiment signal; keeping negations and directional words intact is essential.
4. **Class imbalance handling is essential:** without balanced class weights, classifiers default to predicting Neutral.

Limitations & future work: CPU-only prevented full fine-tuning and made decoder inference infeasible; no hyperparameter optimisation; single dataset. GPU fine-tuning, cross-family ensembles, and temporal analysis are the natural next steps.